

Vektorer som pekare

Vektorer är en gammal datatyp som funnits med i standard C i långa tider. Därför är det inte förvånande att det går att manipulera vektorer även som pekare. I grund och botten är varje vektor en pekare till en minnesadress som har utrymme för ett visst antal element i följd av en viss datatyp. Om man lämnar bort [] på en vektor kan man hantera denna precis som en pekare. Exempelvis:

```
double TalVektor [8];
double * TalPekare;

TalPekare = TalVektor;
// eller
TalPekare = &TalVektor[0];
```

Således är en vektor alltid en pekare! I exemplet ovan är det senare sättet ekvivalent med det första, och med lite eftertanke märker man att det stämmer. `TalVektor` är en pekare och pekar på det första elementet i vektorn, d.v.s. `TalVektor[0]`, och tar man adressen för detta första element får man, just det, `TalPekare`.

Pekararitmetik

Eftersom vi nu har visat att varje vektor är en pekare till minnesadressen för första elementet i en följd, kan vi titta lite vidare på aritmetik med pekare. En pekare är ju i grund och botten ett heltal (`unsigned int`) som pekare på en minnesadress. Vi kan förstås manipulera detta tal med normala aritmetiska operationer, såsom + och -. Vi kan skriva om ovanstående vektorexempel så vi använder pekare istället:

```
#include <iostream>
#include <iomanip>

int main () {

    int Tal [10];

    // iterera och fyll vår vektor med hjälp av vektorn använd som
    // en pekare
    for ( int Index = 0; Index < 10; Index++ ) {
        *(Tal + Index) = Index * Index;
    }

    // deklarera en annan pekare till vektorn
    int * TalPekare = Tal;

    // iterera nu och skriv ut talen
    cout << "Tal   kvadrat" << endl;
    for ( int Index = 0; Index < 10; Index++ ) {
        cout << setw (3) << Index << setw (9) << *TalPekare++ << endl;
    }
}
```

Utskriften är samma som för föregående program. Exemplet kan dock behöva vissa förklaringar. Innan det en pekare pekar på kan användas måste den avrefereras. Det görs t.ex. på raden

```
*(Tal + Index) = Index * Index;
```

Här adderas ett tal till pekaren och sedan avrefereras denna. På så vis kan vi få pekaren att iterera över de olika positionerna i vektorn. Vi måste ha en parentes, eftersom avrefereringsoperatoren har samma precedens som +, så utan parentes skulle vi addera `Index` till det som `Tal` pekar på. En stor skillnad. Senare skapas en pekare `TalPekare` och sätts att peka på `Tal`:

```
int * TalPekare = Tal;
```

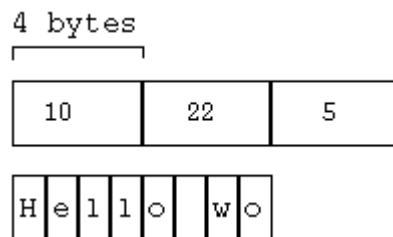
Denna pekare är inte nödvändig men vi får då en chans att visa hur vi kan ha en temporär pekare att iterera över elementen i en vektor med operatoren ++:

```
*TalPekare++
```

Vad som händer här är att ++ fungerar som *postfixinkrementerande*, d.v.s. `TalPekare`:s värde ökas efter att den använts. Användningen i det här fallet är att den avrefereras. Efter användningen ökar ++ värdet på sin operand, alltså `TalPekare` med 1, och sätts således att peka på nästa element i vektorn.

Hur hänger det här ihop då? Den skarpsynte märker att en pekare är en normal `unsigned int` och operatoren ++ bode öka dess värde med 1, men det som `Tal` innehåller har troligtvis en storlek av 4 bytes per element. Operatoren ++ har alltså flyttat pekaren fram 4 minnesadresser!

Figur 9-3. Pekare och minnesadresser



I C++ är operatoren ++ så intelligent så den "vet" vilken datatyp en pekare pekar på, och kan således öka pekarens värde med så mycket som behövs för att flytta till nästa element. I fallet med t.ex. strängar som i bilden ovan flyttar ++ pekaren framåt så mycket som en `char` upptar, d.v.s. normalt 1 byte. Även --, +=, -= o.s.v. fungerar på samma sätt. I princip adderas (eller subtraheras) ett värde från pekaren som är lika stort som `sizeof (datatyp)`. Med hjälp av ++ och -- kan man göra väldigt korta och effektiva iterationer över vektorer med hjälp av pekare. Ibland lönar det sig dock att offra lite effektivitet för att uppnå lättlästa program. Operatoren [] är trots allt lättare att förstå och läsa en direkt pekararitmetik! För den nyfikna kan nämnas att kompilatorer i de flesta fall översätter indexering med [] till pekararitmetik innan själva koden genereras.

Vektorer som funktionsparametrar

Vektorer kan förstås även skickas som parametrar till funktioner. Det finns två olika sätt att skicka vektorer som argument till funktioner, tack vare att vektorer är pekare. Man kan skicka dem som pekare eller som vektorer. Skickas de som pekare som vet mottagarfunktioner inte någonting om vektorns storlek, utan denna måste vara känd på något annat sätt, t.ex. via en annan parameter. Skickas den som en vektor så ges storleken som en del av parameterdefinitionen. Exemplet nedan använder sig av de två olika sätten:

```
#include <iostream>
```

```
void in (int * Tal, int Antal) {
    // iterera och läs in tal
    for ( int Index = 0; Index < Antal; Index++ ) {
        cout << "Ge in tal " << Index + 1 << ": ";
        cin >> *(Tal + Index);
    }
}

void ut (int Tal[3]) {
    // skriv ut vårt tal
    for ( int Index = 0; Index < 3; Index++ ) {
        cout << "Tal " << Index << " är: " << Tal[Index] << endl;
    }
}

int main () {
    int Tal [3];

    // läs in och skriv sedan ut
    in ( Tal, 3 );
    ut ( Tal );
}
```

Funktionen `in()` tar emot vektorn som en pekare. Enbart på bas av parameterdefinitionen vet man inte om det är frågan om en pekare till en enskild `int` eller om det är en pekare till första elementet i en vektor, därför skickas den extra parametern `Antal` med. Funktionen `ut()` däremot tar emot vektorn som en vektor, och då är ju storleken känd. Det senare sättet verkar ju vara mycket bättre, eftersom man då slipper bekymra sig om storlekar o.dyl., men den är även mera begränsad. Till `ut()` kan endast vektorer med tre element skickas, ingenting annat duger, medan `in()` är mycket mera flexibel och kan ta emot vilken storleks vektorer som helst. I båda fallen måste dock datatypen vara densamma hela tiden. Vilket sätt man föredrar beror långt på till vad funktionen skall användas.

[Företgående](#)
Vektorer

[Hem](#)
[Upp](#)

[Nästa](#)
Flerdimensionella vektorer